

MINI PROJECT

ON

Smart Weather Monitoring System with Cloud Integration and Alert Notifications

**BACHELOR OF TECHNOLOGY IN COMPUTER SCIENCE AND
ENGINEERING (Internet of Things)**

Submitted by:

B Sai Swaroop

BU22CSEN0600184

Mythri J

BU22CSEN0600192

Ragavi M

BU22CSEN0600196

Course code: **CSEN4011P**

Cloud-Based – IoT Lab



Department of Computer Science & Engineering,

GITAM SCHOOL OF TECHNOLOGY

GANDHI INSTITUTE OF TECHNOLOGY AND MANAGEMENT

(Deemed to be University)

Bengaluru Campus.

NOVEMBER 2025

Table of Contentss

1. ABSTRACT	3
2. COMPONENTS	3
3. PIN DIAGRAM	4
4. CIRCUIT DIAGRAM.....	5
5. PROCEDURE.....	5
6. CODE	8
7. RESULT	10

1. ABSTRACT

Monitoring weather parameters such as temperature and humidity is essential in various domains, including agriculture, smart homes, industry, and environmental analysis. Traditional weather monitoring systems usually require hardware sensors, microcontrollers, and local storage, making them costly and difficult for beginners to set up. Additionally, many systems lack remote access and automated alert mechanisms, which limits their usefulness in real-time decision-making.

To overcome these limitations, this project proposes a Smart Weather Monitoring System with Cloud Integration and Alert Notifications that operates completely online, eliminating the need for physical hardware. The system uses an IoT simulator to generate temperature and humidity data, which is then uploaded to the Adafruit IO cloud platform. The cloud stores, displays, and continuously monitors the data. When temperature or humidity crosses a predefined threshold, an automatic email alert is sent to the user, enabling instant awareness of abnormal conditions.

This project demonstrates an accessible, low-cost, and scalable approach to IoT-based environmental monitoring. It highlights how real-time cloud integration and automation can be achieved with only simulation tools, making it ideal for academic learning, beginners, and prototype development.

2. COMPONENTS

A. Software Components

1. **Wokwi IoT Simulator** – For generating and visualising virtual sensor readings.
2. **Adafruit IO Cloud Platform** – For data storage, monitoring, and visualisation.
3. **Adafruit IO Triggers** – For sending automatic email notifications.
4. **Web Browser (Chrome/Edge)** – To access cloud dashboards.
5. **Internet Connection** – For communication between the simulator and the cloud.

B. Virtual Hardware Components (Inside the Simulator)

1. ESP32 Virtual Microcontroller
2. DHT22 Virtual Temperature-Humidity Sensor
3. Jumper Wires (Virtual)
4. Power Connections (Virtual)

3. PIN DIAGRAM

ESP32:

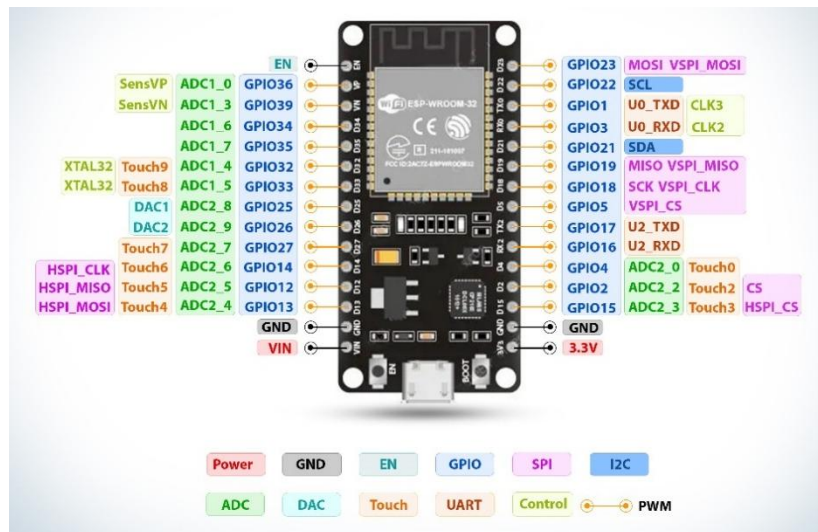


Figure1: Pin Diagram for ESP32 Microcontroller

DHT22:

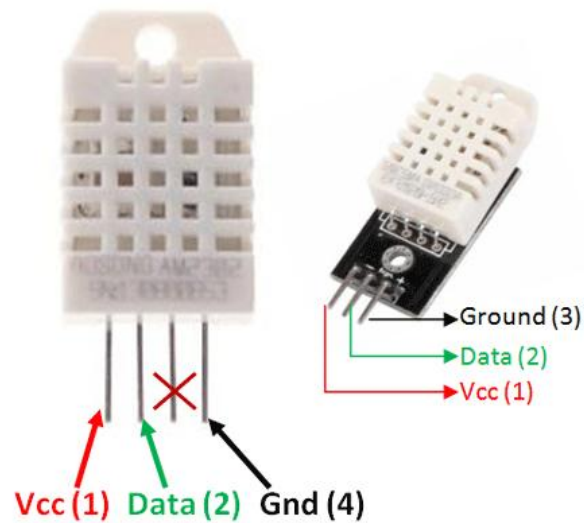


Figure2: Pin Diagram for DHT22 Sensor

4. CIRCUIT DIAGRAM

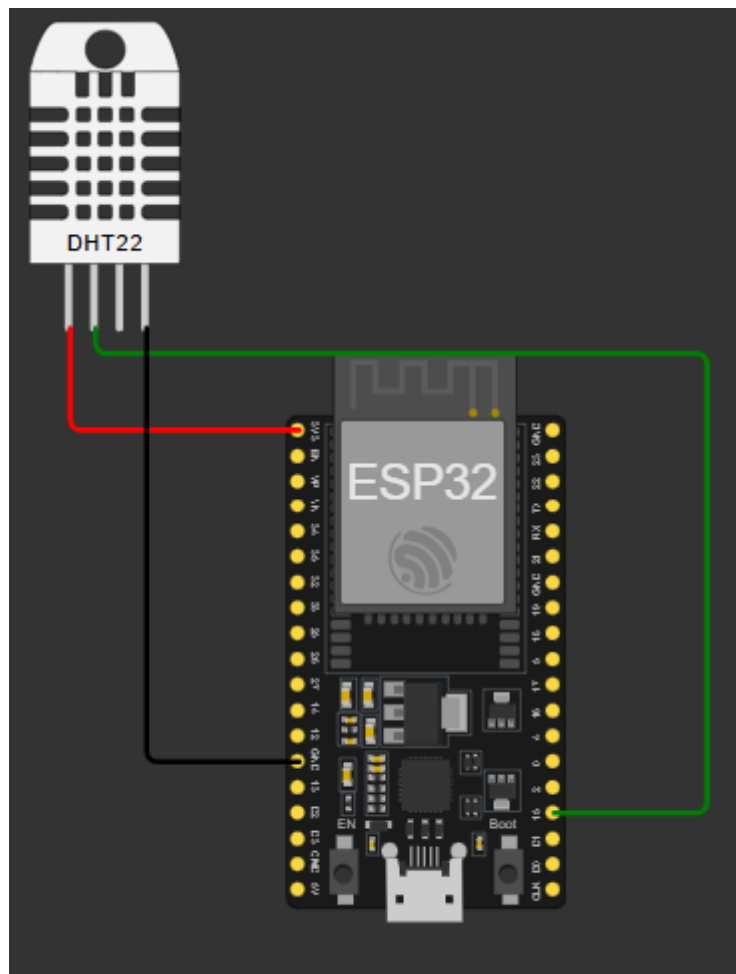


Figure 3: Pin Diagram for ESP32 Microcontroller

5. PROCEDURE

1. Create an Adafruit IO Account

1. Open browser → go to <https://io.adafruit.com>
2. Click **Sign Up**.
3. Enter your **Email, Username, and Password**.
4. Confirm email verification.
5. Log in to the dashboard.

2. Create Feeds for Temperature & Humidity

1. On the left menu → click **Feeds**.
2. Click **New Feed**.
3. Create feed name: **temperature** → Save.
4. Again, click **New Feed**.

5. Create feed name: **humidity** → Save.

(These feeds will store the values coming from the ESP32 simulation.)

3. Get Your AIO Key (Important)

1. Click your profile icon (top right).
2. Select **My Key**.
3. Copy the following:
 - **Username**
 - **Active Key (AIO Key)**
4. Paste these into a notepad (you'll need them for code).

4. Set up Wokwi Simulation

1. Go to <https://wokwi.com/projects/new/esp32>
2. A blank ESP32 project opens.
3. Click the + icon and add:
 - **DHT22 Sensor**
4. Connect the wires:
 - **DHT22 VCC** → **ESP32 3V3**
 - **DHT22 DATA** → **ESP32 GPIO 15**
 - **DHT22 GND** → **ESP32 GND**

5. Run the Simulation

1. Click the **Play** button .
2. The ESP32 will start sending values.
3. Temperature & humidity values will appear in the **Serial Monitor**.

6. Verify Data on Adafruit IO

1. Go to **Adafruit IO** → **Feeds**.
2. Open **temperature** feed → you will see values.
3. Open **humidity** feed → you will see values.

7. Create Cloud Dashboard for Monitoring

1. On the left → click **Dashboard**.
2. Click **New Dashboard** → give a name.
3. Inside dashboard → click + to add widgets:
 - **Line Gauge** → Connect to temperature feed
 - **Gauge** → Connect to humidity feed

Now you have a live monitoring screen.

8. Create Email Alert Notifications (Triggers)

1. Go to **Triggers** (left panel).
2. Click **New Trigger**.
3. Select source feed → **temperature**.
4. Condition:
 - **Greater than** → enter **30**
5. Action:
 - Send an **Email** to your Gmail.
6. Save trigger.

Repeat another trigger for **humidity**:

- Condition: humidity > **80%**

9. Testing Alerts

1. Go back to Wokwi.
2. Click the DHT22 sensor → set custom values.
3. Enter:
 - Temperature: **45°C**
 - Humidity: **80%**
4. ESP32 sends these values to the cloud.
5. Within seconds → Check your email.
6. You will receive:

⚠ Warning: High Temperature Alert!

The temperature in your location is above 30 °C.

⚠ Warning: High Humidity Alert!

The humidity in your location is above 80%.

6. CODE

```
import network
import time
from umqtt.simple import MQTTClient
import dht
from machine import Pin

# -----
# USER SETTINGS
# -----
WIFI_SSID = "Wokwi-GUEST"
WIFI_PASS = ""

AIO_USERNAME = "saiswaroop05"
AIO_KEY = "aio_SSUK185R8Cvp8xyekFPjuV6QfqsI"

AIO_FEED_TEMP = AIO_USERNAME + "/feeds/temperature"
AIO_FEED_HUM = AIO_USERNAME + "/feeds/humidity"

# -----
# DHT22 SENSOR
# -----
dht_pin = Pin(15, Pin.IN)
sensor = dht.DHT22(dht_pin)

# -----
# CONNECT TO WIFI
# -----
def connect_wifi():
    wifi = network.WLAN(network.STA_IF)
    wifi.active(True)
    wifi.connect(WIFI_SSID, WIFI_PASS)

    print("Connecting to WiFi...", end="")
    while not wifi.isconnected():
        print(".", end="")
        time.sleep(0.5)
    print(" Connected!")
    print(wifi.ifconfig())

# -----
# MQTT CONNECT
# -----
def connect_mqtt():
    client = MQTTClient(
        client_id="esp32_dht22",
        server="io.adafruit.com",
        user=AIO_USERNAME,
```



```

    password=AIO_KEY,
    ssl=False
)
client.connect()
print("Connected to Adafruit IO!")
return client

# -----
# MAIN PROGRAM
# -----
connect_wifi()
mqtt_client = connect_mqtt()

while True:
    try:
        sensor.measure()
        temperature = sensor.temperature()
        humidity = sensor.humidity()

        print("Temp:", temperature, "°C | Hum:", humidity, "%")

        # Publish to Adafruit IO
        mqtt_client.publish(AIO_FEED_TEMP, str(temperature))
        mqtt_client.publish(AIO_FEED_HUM, str(humidity))

        print("Published to AIO!")

    except Exception as e:
        print("Sensor error:", e)

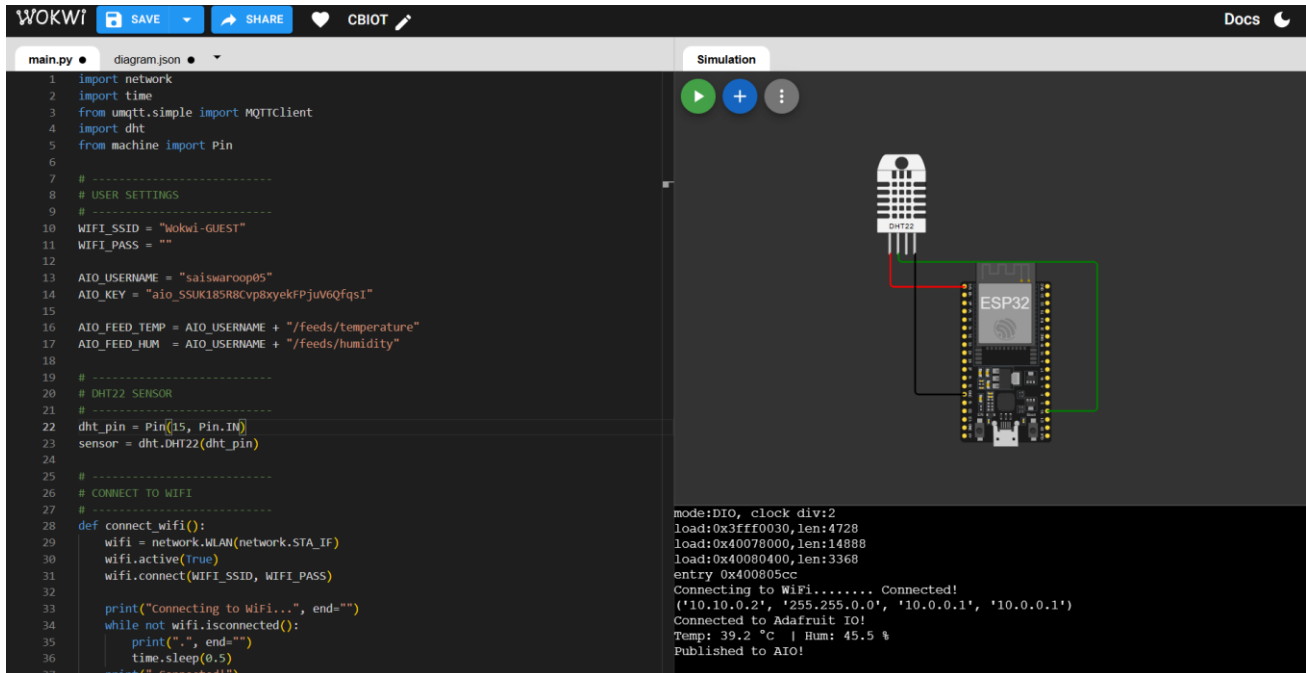
    time.sleep(15) # Read every 5 seconds

```

7. RESULT

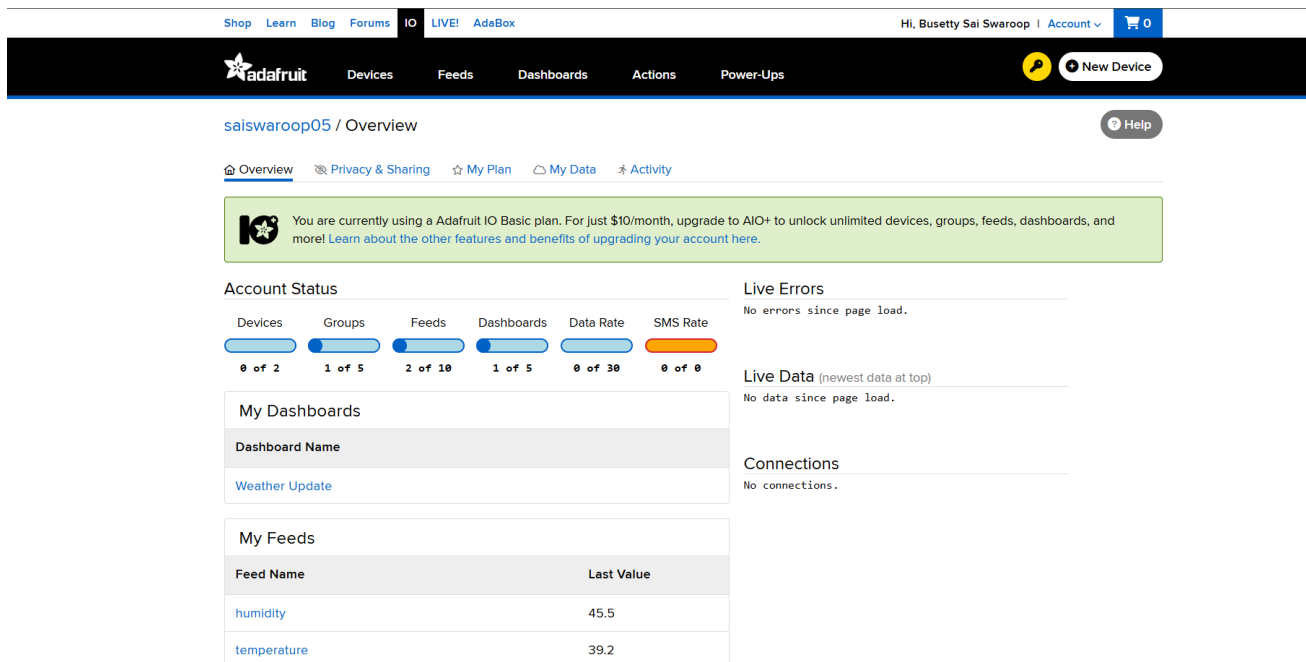
- The system successfully simulated temperature and humidity data using Wokwi.
- Real-time data was transmitted to Adafruit IO cloud without any errors.
- The dashboard displayed live graphs, gauges, and feed updates.
- Email alerts were sent instantly when threshold limits were exceeded.
- The project meets all objectives:

- Simulation



The screenshot displays the Wokwi web interface for simulating an ESP32 microcontroller. On the left, a Python script is shown, which imports necessary libraries, sets up WiFi credentials, and configures an Adafruit IO feed. It uses a DHT22 sensor to read temperature and humidity, then publishes this data to the cloud. On the right, the 'Simulation' window shows a visual representation of the ESP32 board connected to a DHT22 sensor module. Below the simulation, a log window shows the execution output, including successful WiFi connection and data publishing to Adafruit IO with values like Temp: 39.2 °C and Hum: 45.5 %.

- Cloud storage



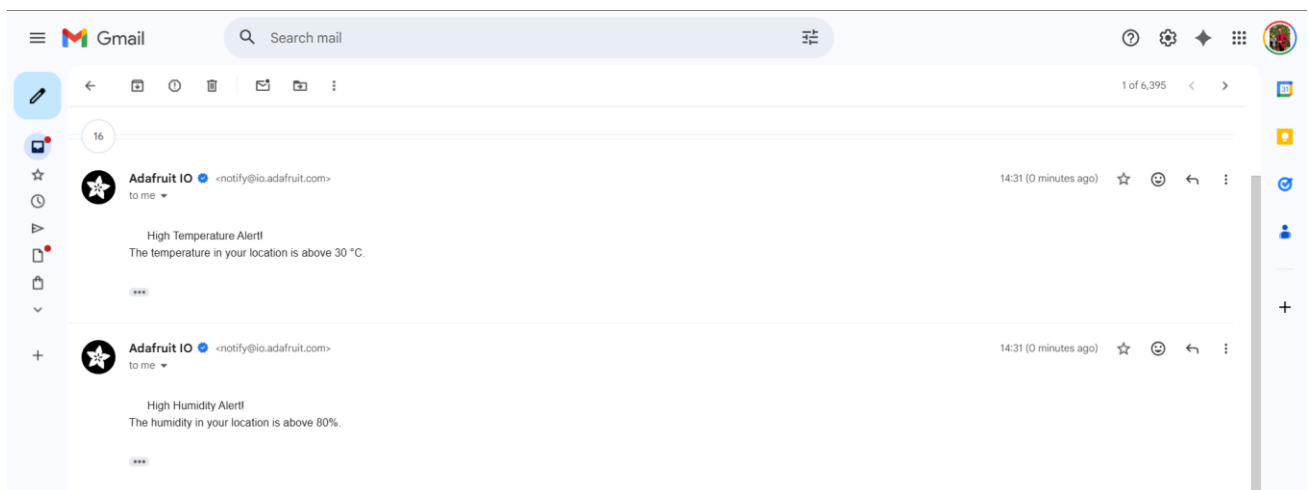
The screenshot shows the Adafruit IO dashboard for the user 'saiswaroop05'. The top navigation bar includes links to Shop, Learn, Blog, Forums, and IO. The dashboard is divided into several sections: Account Status, Live Errors, Live Data, and Connections. The Account Status section shows usage for Devices, Groups, Feeds, Dashboards, Data Rate, and SMS Rate. The Live Data section shows the latest data for the 'humidity' and 'temperature' feeds. The Connections section shows no active connections.

Feed Name	Last Value
humidity	45.5
temperature	39.2

- Data visualisation



- Automated alerts



Outcome:

The Smart Weather Monitoring System was successfully implemented online, demonstrating IoT concepts without the need for physical sensors or hardware.